

Projet : Conception d'une Serre Intelligente

FISA S3E

Année : 2025–2026

Groupe étudiants :

ALEMANY Paul

CHASTANET Bastien

JOUFFREAU Lucas

Tuteur :

Mohamed AIT-BRAHIM

Sommaire

1. Résumé 4
2. Table des matières 4
3. Table des figures 4
4. Introduction 5
5. État de l'art 6
6. Présentation du projet 7
6.1. Synthèse des fonctions de service 8
6.2. Synoptique du système 8
7. Analyse des choix techniques 9
7.1. Analyse économique et choix des composants 9
7.2. Justification des choix technologiques 9
7.2.1 Capteurs et traitement du signal 9
7.2.2 Pilotage des actionneurs 10
7.2.3 Architecture logicielle 10
7.2.4 Contraintes et sécurité 10
8. Conception et modélisation du système 11
8.1. Architecture globale 11
8.2. Spécifications du système (Exigences) 11
8.3. Schémas de câblage et interface matérielle 12
8.3.1 Câblage général 12
8.3.2 Câblage actionneurs 13
8.3.3 Câblage écran LCD 13
8.4. Modélisation logicielle 14
8.4.1 Mode automatique 14
8.4.2 Mode manuel 16
8.5. Description détaillée du fonctionnement 17
9. Réalisation et implémentation 19
9.1. Montage matériel 19
9.2. Implémentation logicielle 20
9.2.1 Structure du projet 22
9.2.2 Optimisation des ressources 24

9.2.3 Logique de contrôle-commande 24
9.2.4 Gestion des erreurs 25
9.3. Tests et validation 26
9.3.1 Validation de la chaîne d'acquisition (SR-GH-001, 002) 26
9.3.2 Validation des boucles de régulation (SR-GH-004, 005, 006, 007) 26
9.3.3 Validation de l'IHM et de l'interactivité (SR-GH-003, 008, 009) 27
9.3.4 Validation de l'IHM et de l'interactivité (SR-GH-010) 27
9.3.5 Test de sécurité et diagnostic (SR-GH-011) 27
10. Gestion de projet / management 28
11. Analyse critique 28
12. Bilan du projet 28
13. Conclusion 28
14. Glossaire 28
Annexes 28
Specification système 30
Design écran LCD 33

1. Résumé

Ce projet, réalisé dans le cadre du module Microcontrôleur, porte sur la conception et le développement d'un système embarqué de gestion pour une serre intelligente.

L'objectif principal est la régulation autonome de la température ambiante et de l'humidité du sol à l'aide d'un microcontrôleur type STM32.

Le système intègre une chaîne d'acquisition complète (capteurs analogiques température et humidité) et des algorithmes de contrôle-commande variés :

- Une régulation Tout-Ou-Rien avec hystérésis pour la ventilation,
- Un asservissement linéaire par modulation de largeur d'impulsion (PWM) pour l'irrigation,
- Une interface humain-machine (IHM) composée d'un écran LCD et de boutons-poussoirs permet un paramétrage dynamique des seuils et un mode de pilotage manuel,
- Une couche de diagnostic de sécurité (Error Handler) garantissant la fiabilité du prototype.

Les résultats valident un système autonome, robuste et conforme aux exigences du cahier des charges initial.

2. Table des matières

3. Table des figures

4. Introduction

L'agriculture moderne fait face à des défis sans précédent :
- optimisation des ressources hydriques,
- maîtrise de la consommation énergétique,
- adaptation aux aléas climatiques.

Dans ce contexte, les serres intelligentes s'imposent comme une solution technologique clé pour garantir un environnement de culture optimal tout en minimisant l'intervention humaine.

Le projet présenté dans ce rapport s'inscrit dans le cadre du module "Microcontrôleur".
L'objectif était de concevoir, en une durée limitée de 38 heures, un démonstrateur fonctionnel de serre automatisée capable de réguler de manière autonome la température et l'humidité du sol.

Ce travail a nécessité une approche pluridisciplinaire mêlant :
- Électronique (chaîne d'acquisition et pilotage d'actionneurs),
- Automatique (asservissement PWM et régulation à hystérésis),
- Informatique embarquée (développement en langage C sur architecture ARM Cortex).

Ce rapport détaille les étapes de cette réalisation, depuis l'analyse des besoins jusqu'à la validation expérimentale du prototype, en mettant en avant les choix techniques qui ont permis d'aboutir à un système fiable et ergonomique.

5. État de l'art

Les serres automatisées représentent aujourd'hui une réponse technologique aux enjeux de l'agriculture moderne dite 4.0 et de l'autoproduction alimentaire.

Elles permettent de surveiller et de contrôler des paramètres essentiels tels que l'humidité du sol, la température ou la luminosité, en s'appuyant sur des systèmes embarqués compacts et économes en énergie.

Plusieurs travaux de recherche mettent en évidence l'intérêt de ces dispositifs.

En 2021, Vinoth Kumar P., K.C. Ramya, Abishek J.S., Arundhathy T.S., Bhavvya B. et Gayathri V. ont présenté lors de l'International Conference on Signal Processing and Communication (ICSPC) un projet intitulé « *Smart Garden Monitoring and Control System with Sensor Technology » (1)*. Leur prototype démontre la faisabilité d'un suivi automatisé de l'humidité du sol et de la température, associé à un contrôle automatique de l'irrigation.

Solutions technologiques actuelles :

Le marché et la communauté open-source s'articulent principalement autour de trois piliers :

- Microcontrôleurs : Arduino (accessibilité), ESP32 (connectivité Wi-Fi) et STM32 (performance industrielle et basse consommation),
- Capteurs : Sondes capacitives (DHT22) pour l'humidité (plus durables que les versions résistives) et capteurs analogiques de précision pour la température,
- Actionneurs : Pompes submersibles et ventilateurs à flux axial.

Limites identifiées et positionnement du projet :

Bien que ces solutions soient performantes, elles présentent des lacunes auxquelles ce projet tente de répondre.

Dépendance aux bibliothèques haut niveau :

- De nombreux projets utilisent des couches logicielles opaques (type Arduino),
- Dans notre projet, l'utilisation du C bas niveau et des drivers HAL sur STM32 garantit une maîtrise totale du cycle d'instruction et de la consommation mémoire.

Stabilité des mesures :

- Les systèmes basiques sont sensibles au bruit électronique,
- L'implémentation d'un filtre numérique à moyenne glissante dans notre solution permet de s'affranchir de cette limite.

Absence de diagnostic :

- La plupart des prototypes ne gèrent pas la défaillance d'un composant,
- Notre intégration d'un module de gestion d'erreurs (Error Handler) apporte une dimension de sécurité industrielle souvent absente des projets pédagogiques.

(1) Vinoth Kumar P., K.C. Ramya, Abishek J.S., Arundhathy T.S., Bhavvya B., Gayathri V., Smart Garden Monitoring and Control System with Sensor Technology, ICSPC, Coimbatore, 2021.

6. Présentation du projet

Le projet de conception d'une serre intelligente s'inscrit dans le cadre du module Microcontrôleur de quatrième année du cycle ingénieur.

L'objectif pédagogique est de nous permettre de mettre en pratique les compétences acquises en programmation bas niveau et en conception de systèmes embarqués.

Sur le plan technique, il vise à démontrer la capacité d'un microcontrôleur à collecter des données environnementales, à prendre des décisions en fonction de seuils configurés et à piloter automatiquement des actionneurs afin de maintenir des conditions optimales pour la culture dans une serre.

Le cahier des charges fixe plusieurs exigences précises :

- Le système doit être capable de mesurer l'humidité du sol et la température ambiante grâce à des capteurs adaptés.
- Il doit également piloter une pompe pour l'irrigation et un ventilateur pour la régulation thermique.
- Une interface locale composée d'un écran et de boutons poussoirs doit permettre à l'utilisateur de consulter les données et d'interagir directement avec le système.
- L'automatisation repose sur la comparaison des valeurs mesurées avec des seuils configurables, ce qui permet au microcontrôleur de déclencher les actions nécessaires sans intervention humaine.

Le projet doit respecter plusieurs contraintes technologiques :

- Le système doit fonctionner sur un microcontrôleur de type STM32 ou ESP32, sans recours à une couche d'abstraction comme Arduino.
- Les communications entre les différents composants doivent se limiter à des interfaces simples telles que GPIO, ADC/DAC ou UART.
- L'ensemble doit être alimenté par une source unique en 5 ou 12 volts et conçu de manière propre et fiable pour assurer la robustesse et la réplicabilité du prototype.

L'absence de framework type Arduino nous a imposé une gestion directe des registres via la bibliothèque HAL (Hardware Abstraction Layer), garantissant ainsi une empreinte mémoire optimisée et une meilleure compréhension de la gestion des interruptions et des timers du microcontrôleur.

En résumé, ce projet combine des objectifs pédagogiques et techniques clairs, un cahier des charges précis et des contraintes réalistes, afin de permettre aux étudiants de concevoir une serre intelligente fonctionnelle et représentative des enjeux actuels de l'automatisation agricole.

Le livrable final comprend ce rapport ainsi que le code du projet et une présentation du fonctionnement du projet.

6.1.Synthèse des fonctions de service

Afin de répondre au cahier des charges, le système a été décomposé en quatre fonctions principales :

Fonction	Description technique	Composant associé
Acquérir	Mesure des grandeurs physiques analogiques	Sonde température : TMP235 Sonde humidité : SEN0193
Traiter	Analyse des données et calcul des lois de commande	Microcontrôleur : stm32g031k8
Communiquer	Interface utilisateur et réglage des seuils	Ecran LCD : HD44780U 2x16 Boutons-poussoirs
Agir	Régulation physique de l'environnement	Pompe (PWM) : FIT0910 Ventilateur : MF40100V2 Via carte moteur : MotoDriver2

6.2.Synoptique du système

Le synoptique suivant illustre l'architecture modulaire du prototype.

Le **STM32G031U6** centralise les données analogiques via son **ADC** et gère les interactions utilisateurs par des interruptions externes (**EXTI**).

Le pilotage des actionneurs est différencié : un signal **PWM** assure une irrigation progressive, tandis que le ventilateur est commandé en Tout-Ou-Rien via le driver de puissance.

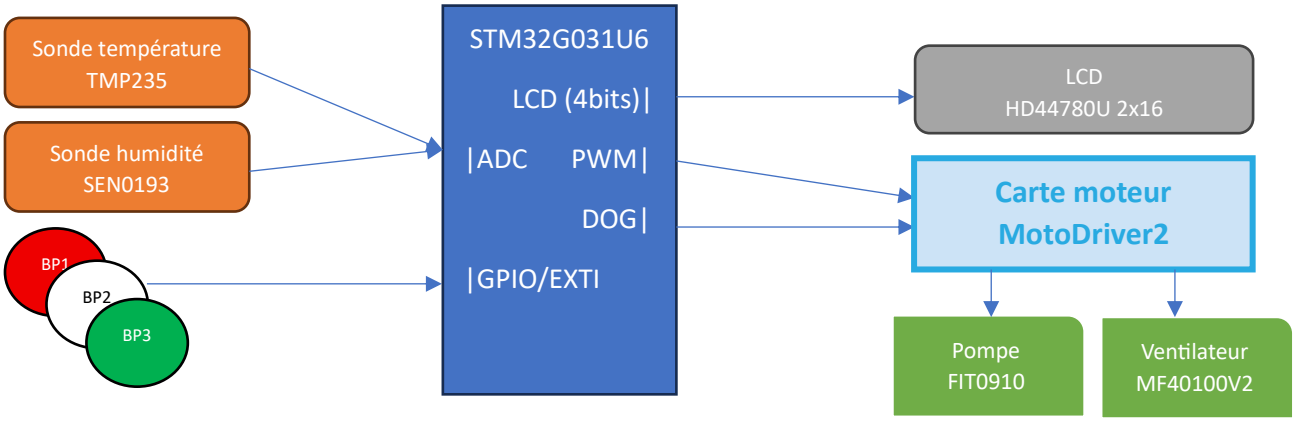


Figure 1 Synoptique général du système Greenhouse.

7. Analyse des choix techniques

7.1.Analyse économique et choix des composants

Le choix du microcontrôleur s'est porté sur un STM32. Ce composant est largement disponible sur le marché, avec des cartes de développement comme les Nucleo ou les BluePill proposées à des prix accessibles, généralement compris entre 10 et 25 euros.

Pour la mesure de l'humidité du sol, un capteur capacitif de type SEN0193 a été retenu.
Ce capteur est fiable et résistant à la corrosion, contrairement aux capteurs résistifs plus fragiles. Son prix se situe autour de 5 à 10 euros, ce qui en fait une solution économique et durable.

La température est mesurée à l'aide d'un capteur TMP235, très répandu dans les projets embarqués. Ce capteur combine précision et simplicité d'intégration, pour un coût moyen de 5 euros.

L'affichage des données locales est assuré par un écran LCD 2x16. Ce type d'écran est largement utilisé dans les projets étudiants en raison de sa simplicité d'utilisation et de son faible coût, généralement inférieur à 10 euros. Il permet de répondre à l'exigence d'une interface locale tout en restant dans une enveloppe budgétaire limitée.

Pour le pilotage des actionneurs, une pompe à eau et un ventilateur ont été choisis. La pompe, de type FIT0910, est disponible pour un prix compris entre 10 et 15 euros, tandis que le ventilateur MF40100V2 se situe autour de 5 à 10 euros.

La carte MotoDriver2, indispensable pour alimenter et protéger ces actionneurs, est proposée à un prix moyen de 10 à 15 euros.

L'interface utilisateur est complétée par des boutons poussoirs, dont le coût est négligeable (moins d'un euro par unité). L'ensemble du système est alimenté par une source unique en 5 ou 12 volts, via un bloc secteur ou une batterie externe, dont le prix varie entre 10 et 20 euros selon la solution retenue.

7.2.Justification des choix technologiques

7.2.1 Capteurs et traitement du signal

Le choix des composants ne s'est pas limité au coût, mais à la pertinence de leur interface électronique :

- Humidité (SEN0193) : Le choix d'un capteur capacitif est stratégique. Contrairement aux modèles résistifs, il n'y a aucune électrolyse entre les électrodes, ce qui évite la dégradation des mesures dans le temps.
    - Aspect technique : La sortie est analogique, traitée par l'ADC du STM32 sur 12 bits. Un filtre à moyenne glissante a été implémenté en C pour stabiliser la mesure contre le bruit de commutation de l'alimentation.

- Température (TMP235) : L'utilisation d'une sonde analogique linéaire (10mV/°C) permet une lecture directe et rapide sans protocole complexe, facilitant l'implémentation de la loi de commande.

7.2.2 Pilotage des actionneurs

Le pilotage des moteurs nécessite une distinction entre puissance et commande :

Interface de puissance : Le driver MotoDriver2 (Pont en H).

Il permet d'isoler le circuit logique (3.3V) de la puissance (5V/12V) et de protéger le STM32 contre les retours de courants inductifs des moteurs.

Modulation de Largeur d'Impulsion (PWM) : Pour la pompe, nous avons choisi un pilotage par PWM. Cela permet de varier le débit d'eau de manière fluide, évitant ainsi les chocs hydrauliques et permettant un arrosage plus précis par rapport à un simple ON/OFF.

Régulation par Hystérésis : Pour le ventilateur, une logique à hystérésis a été retenue afin d'éviter les commutations trop fréquentes autour du seuil de température, prolongeant ainsi la durée de vie du matériel.

7.2.3 Architecture logicielle

L'architecture a été pensée pour être modulaire et évolutive.

Séparation des couches : Le code est divisé en modules indépendants (Hardware Abstraction vs Logic Application).

Optimisation mémoire : Nous avons développé nos propres fonctions d'affichage pour éviter l'utilisation de bibliothèques standards trop gourmandes en ressources, maximisant ainsi l'autonomie et la rapidité du système.

7.2.4 Contraintes et sécurité

La fiabilité est assurée par le module Error Handler.

Ce "Watchdog" logiciel surveille la cohérence des mesures (ex: une température qui ne baisse pas malgré le ventilateur actif).

En cas de défaut, le système bascule en mode sécurité avec une alerte visuelle LED et affichage LCD.

8. Conception et modélisation du système

8.1.Architecture globale

Le système est divisé en trois couches telles que décrites dans le synoptique suivant.

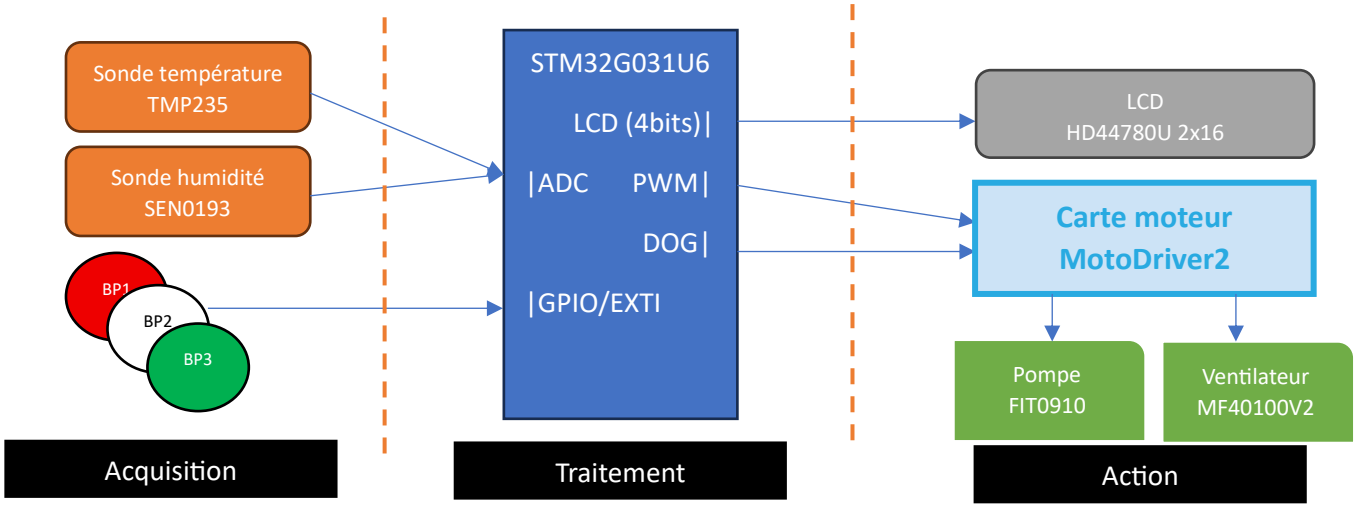


Figure 2 Architecture globale

8.2.Spécifications du système (Exigences)

Le système est intégralement décrit dans le §Annexes - Specification système.

Spécification synthétique :

Exigence	Objet	Rationale
SR-GH-001/002	Acquisition	Mesure de la Température (°C) et de l'Humidité (%) via ADC.
SR-GH-004/005	Monitoring	Comparaison des mesures par rapport aux consignes utilisateur.
SR-GH-008/011	IHM	Affichage multi-pages (Menu 1 à 5) et indicateurs d'état (OK/Défaut).
SR-GH-003	Interactivité	Modification des seuils en temps réel sans reprogrammation.

8.3.Schémas de câblage et interface matérielle

8.3.1 Câblage général

Le STM32 est le cœur de calcul, les échanges avec le monde extérieur sont réalisés de la sorte :

- Entrées Analogiques : Température sur PA0 (ADC1_IN0) et Humidité sur PA1 (ADC1_IN1).
- Interface Utilisateur : 3 boutons-poussoirs sur PA7, PA6 et PA11.
- L'écran LCD est piloté sur un bus de données 4 bits (PB0, PB1, PB2, PB8) avec signaux de contrôle sur PB6 (RS) et PB7 (EN).
- Puissance :
  - Utilisation du timer TIM16_CH1 sur PB4 pour générer le signal PWM de la pompe.
  - Le ventilateur est piloté en GPIO standard sur PB3. Le tout transite par la carte de puissance SBC-MotorDriver2.

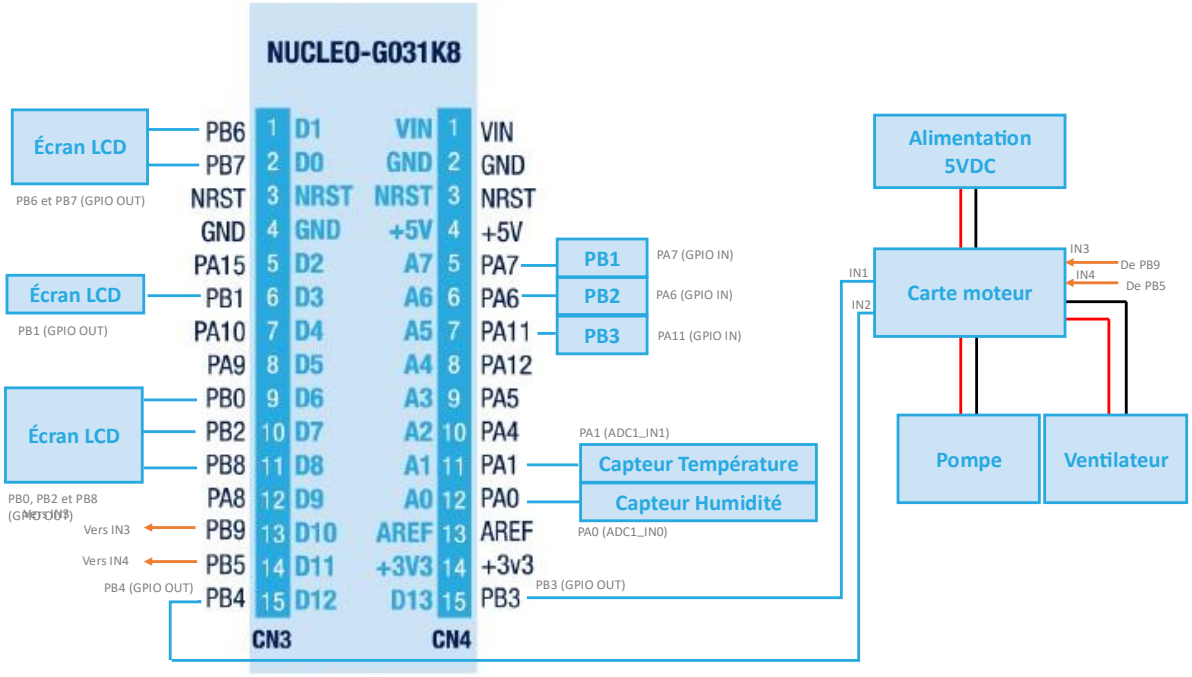


Figure 3 Câblage général

8.3.2 Câblage actionneurs

La carte moteur SVC-MotorDriver2 permet de séparer la partie puissance des actionneurs (5-12VDC) de la partie information (3.3VDC).

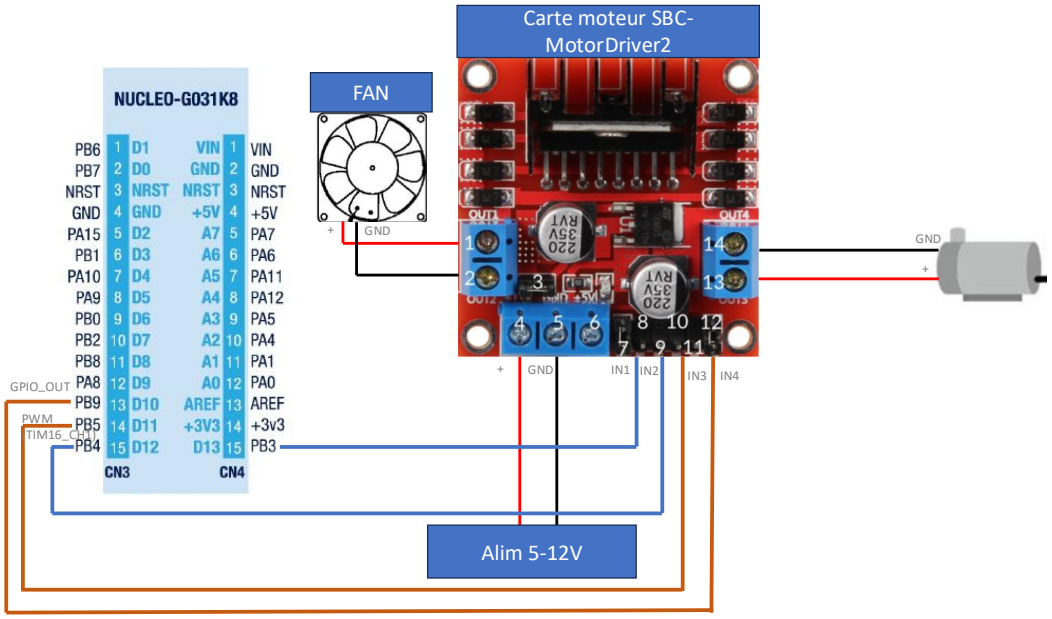


Figure 4 câblage actionneurs

8.3.3 Câblage écran LCD

L'écran LCD est piloté sur un bus de données 4 bits (PB0, PB1, PB2, PB8) avec signaux de contrôle sur PB6 (RS) et PB7 (EN).

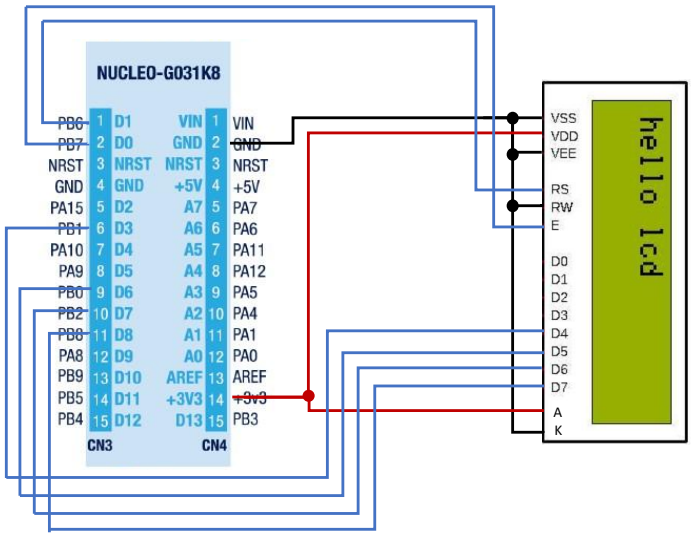


Figure 5 câblage écran LCD

8.4.Modélisation logicielle

Le système a été pensé pour pouvoir fonctionner en deux modes distincts, Automatique et Manuel.

En automatique, c'est le système asservi qui régule la température et l'humidité en fonction de consignes.

En Manuel, c'est l'utilisateur qui active ou éteint les actionneurs.

8.4.1 Mode automatique

En mode automatique, les systèmes de gestion de température et d'humidité sont gérés en fonction de consignes utilisateur.

Régulation de la température (hystérésis) :

Suivant la consigne Tcons, si la température mesurée Tmes est au-dessus de Tcons alors le ventilateur se met en fonctionnement.

Le ventilateur fonctionne en « Marche/Arrêt » il est soit allumé soit éteint.

Pour éviter des ON/OFF intempestifs quand Tmes est très proche de Tcons, le point d'arrêt du ventilateur se fait légèrement sous la consigne à Tcons-0.5°C.

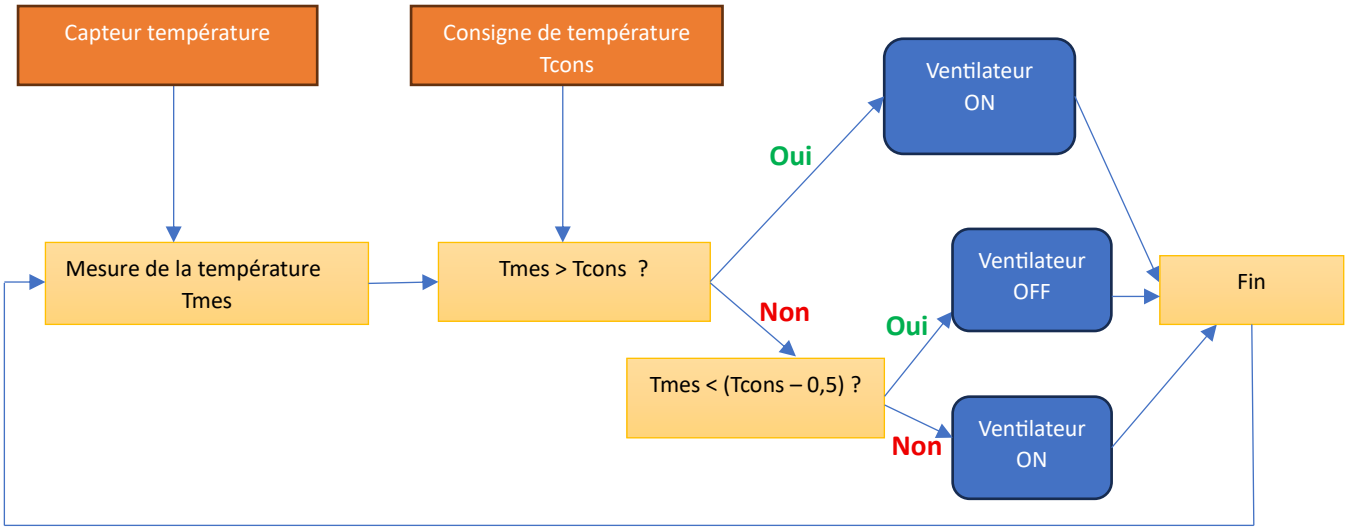


Figure 6 Ventilation automatique

Gestion de l'humidité (Rampe PWM):

En mode automatique, le système utilise une régulation par Modulation de Largeur d'Impulsion (**PWM**). Contrairement au ventilateur, la pompe varie sa vitesse.

Plus l'écart entre l'humidité réelle et la consigne est grand, plus le signal PWM est élevé pour arroser efficacement.

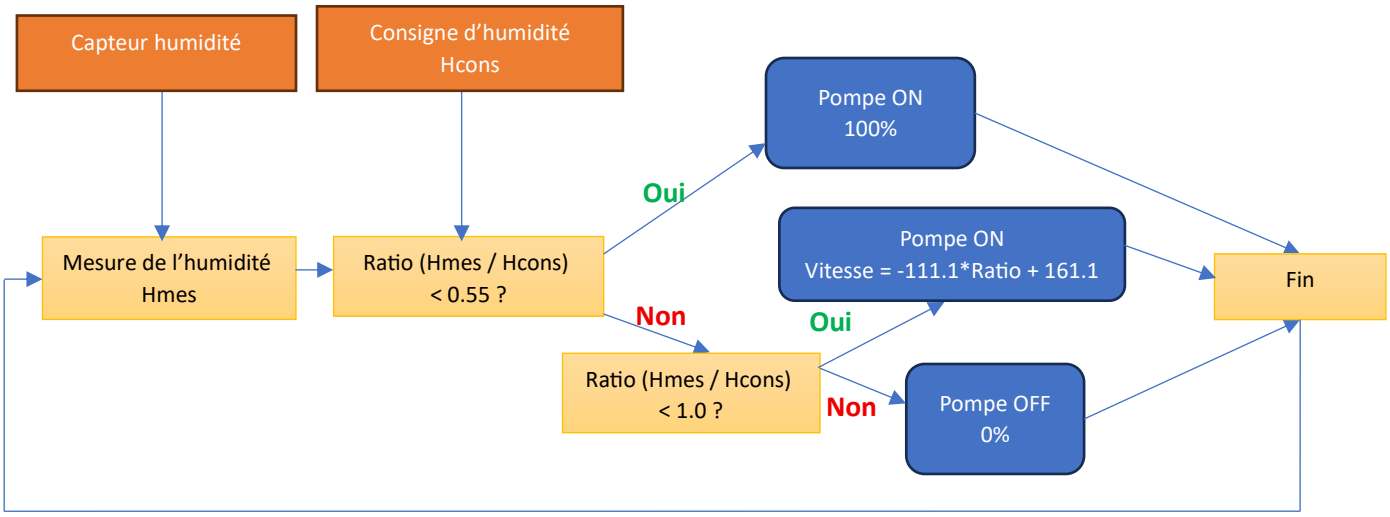


Figure 7 Pompe automatique

La vitesse de la pompe est calculée dynamiquement selon l'écart entre l'humidité mesurée et le seuil de consigne.

Zone de puissance maximale (100%) : Si l'humidité descend en dessous de 55% de la consigne, la pompe tourne à plein régime pour réagir vite.

Zone de régulation linéaire : Entre 55% et 100% de la consigne, la vitesse diminue linéairement suivant l'équation y = -111,1x + 161,1.

Zone d'approche douce (Entre 55% et 50% de la consigne) : Juste avant d'atteindre la consigne, la pompe maintient un débit réduit de 50% pour éviter de détremper le sol par inertie.

Cette rampe permet de réduire l'usure mécanique de la pompe en évitant les arrêts/redémarrages brutaux.

Elle permet également d'éviter l'overshoot (dépasser la consigne d'humidité) grâce au ralentissement

à l'approche du seuil.

La courbe ci-dessous modélise la vitesse de la pompe (en %) en fonction du rapport Hmes/Hcons.

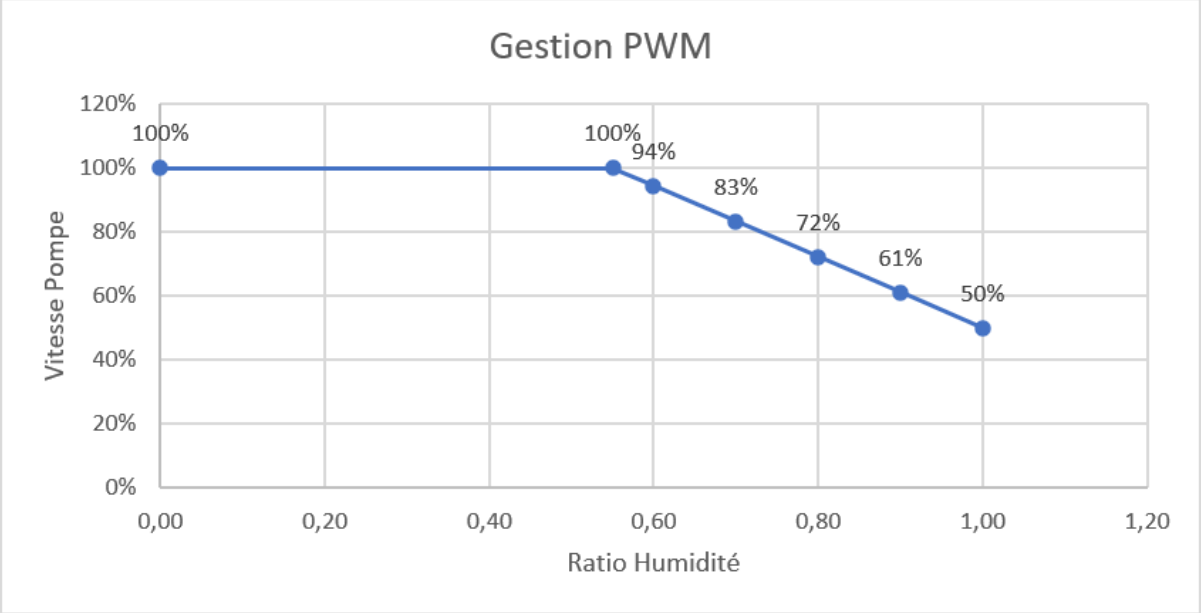


Figure 8 Gestion PWM de la Pompe

8.4.2 Mode manuel

En mode manuel, les systèmes de gestion de température et d'humidité sont activés par l'opérateur. Les sécurités automatiques d'arrêt de la ventilation et/ou de la pompe sont désactivées. L'utilisateur est responsable du refroidissement et de l'humidification.

Gestion de la température :

Le ventilateur fonctionne en « Marche/Arrêt » il est soit allumé soit éteint.

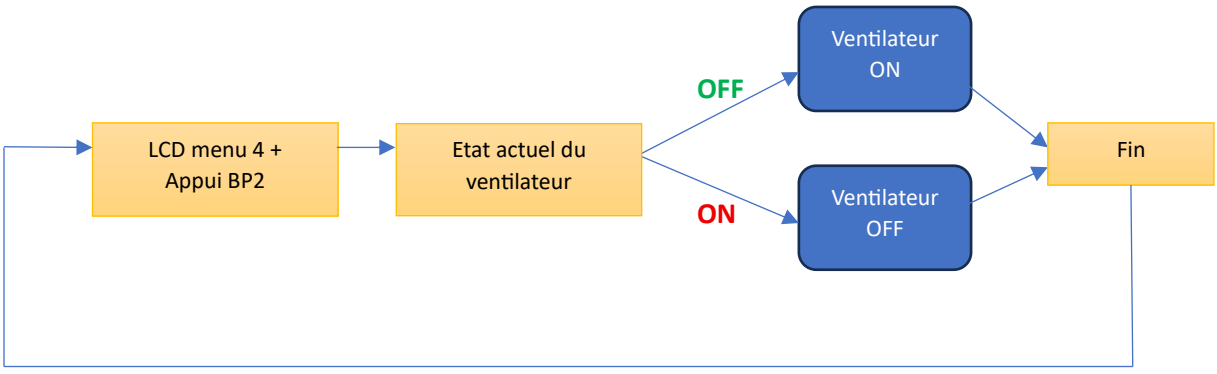


Figure 9 Ventilation manuelle

Gestion de l'humidité :

L'activation manuelle de la pompe se fait à vitesse fixe (100%).

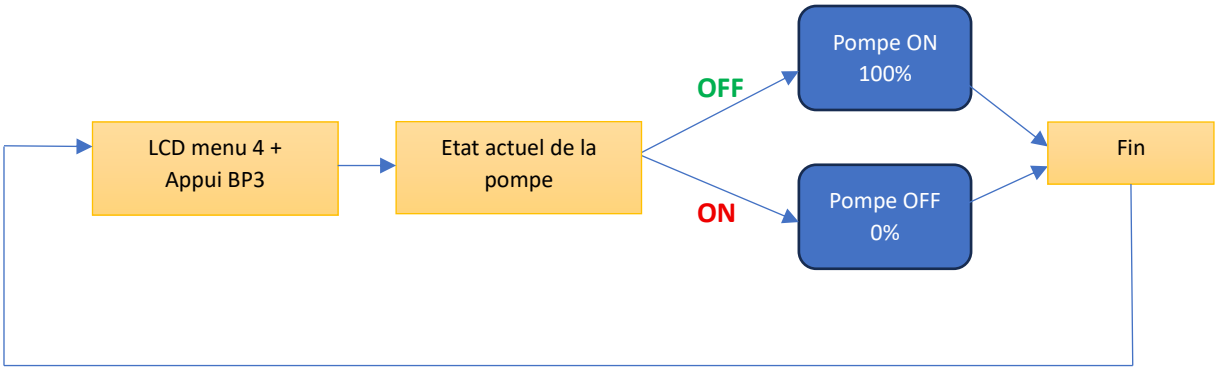


Figure 10 Pompe manuelle

8.5.Description détaillée du fonctionnement

Le fonctionnement du système Greenhouse repose sur une architecture à division temporelle. Le microcontrôleur scrute les capteurs, met à jour les lois de régulation, puis rafraîchit l'interface utilisateur. Le comportement du système peut être décrit comme ceci:

A. La machine d'états de l'IHM (Gestion des Menus)

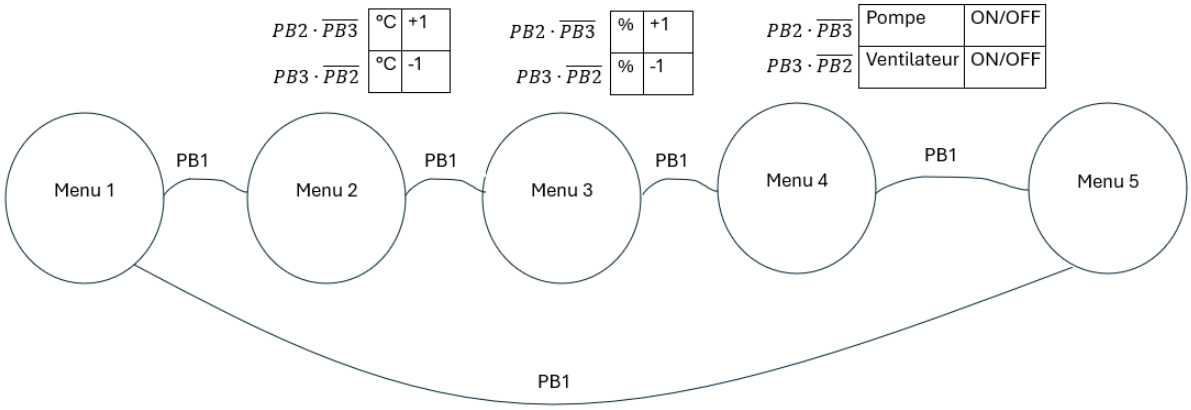


Figure 11 MAE gestion de l'affichage

Pour répondre à l'exigence d'interactivité (SR-GH-003), l'interface LCD est organisée en 5 pages contextuelles, sélectionnables via les boutons-poussoirs. Cette structure permet une navigation intuitive malgré la taille restreinte de l'écran (2x16) :

- Menu 1 (Monitoring) : Affichage dynamique des mesures brutes traitées (T° en °C et H en %).
- Menus 2 & 3 (Configuration) : Permettent à l'opérateur de modifier les seuils critiques.

  L'appui sur les boutons "Plus" ou "Moins" incrémente les variables de consigne stockées en mémoire.

- Menu 4 (Pilotage Manuel) : Mode de forçage permettant de tester les actionneurs indépendamment des capteurs (utile pour la maintenance).

- Menu 5 (Diagnostic) : Affiche l'état du système.

Le design des différents menu est donné en §Annexes - Design écran LCD.

B. Algorithmes de Régulation Automatisée

La régulation s'active dès que le système est en mode automatique. Deux logiques distinctes sont appliquées :

1- Régulation Thermique (Ventilation) : Basée sur une logique de type "Tout-Ou-Rien" avec hystérésis. Le ventilateur s'active si la température dépasse le seuil défini (ex: 25°C) et ne se coupe que lorsqu'elle repasse sous une limite de sécurité (ex: 25°C – 0.5°C). Cette marge évite les micro-déclenchements nuisibles au moteur.

2- Régulation Hydrique (Irrigation) : Utilise une commande proportionnelle via le signal PWM (TIM16_CH1). Si l'humidité descend sous le seuil (SR-GH-007), la pompe s'active. La vitesse de pompage est ajustée dynamiquement pour garantir un apport d'eau progressif.

C. Surveillance et Sécurité (Error Handler)

Conformément à l'exigence SR-GH-011, une couche de surveillance logicielle tourne en arrière-plan. Elle compare l'état des actionneurs avec l'évolution des mesures :

- Si la pompe est active mais que l'humidité ne remonte pas après 15 secondes, le système lève une erreur "Défaut Pompe" (interprété comme un réservoir vide).

- Si une erreur est détectée, la LED de statut passe en mode clignotement rapide et le message correspondant est envoyé sur le Menu 5 du LCD.

9. Réalisation et implémentation

9.1. Montage matériel

Détails du prototype

Photos (optionnel mais conseillé)

A FAIRE !!! PRENDRE DES PHOTOS pour PREUVE

9.2.Implémentation logicielle

Le développement logiciel a été réalisé en langage C sous l'environnement STM32CubeIDE.

Le code est disponible dans un repo GITHUB dans sa branche « main ».

Pour le télécharger :

1- Demander l'accès au projet au détenteur du projet (Lucas Jouffreau)
2- Sur une machine où GIT est installé, rentrer les commandes suivantes
   a. git clone https://github.com/Lucas-byte31/Greenhouse.git
   b. cd Greenhouse // aller sur le dossier cloné
   c. git checkout main // s'assurer que vous êtes bien sur la branche main

L'architecture a été pensée pour être modulaire, séparant la gestion des périphériques (Drivers) de la logique de contrôle (Application).

Le flux de contrôle du système est centralisé dans le fichier main.c.

Afin de garantir une réactivité optimale, nous avons évité les fonctions bloquantes.

Le programme suit un cycle déterministe : il sollicite le module capteurs.c pour obtenir des données stables, exécute les algorithmes de régulation, puis délègue la gestion de l'affichage au lcd_page_manager.c.

Cette séparation stricte des responsabilités permet, par exemple, de modifier la logique de l'écran sans impacter la précision de la régulation thermique.

La figure suivante présente l'architecture logicielle ainsi que son control flow.

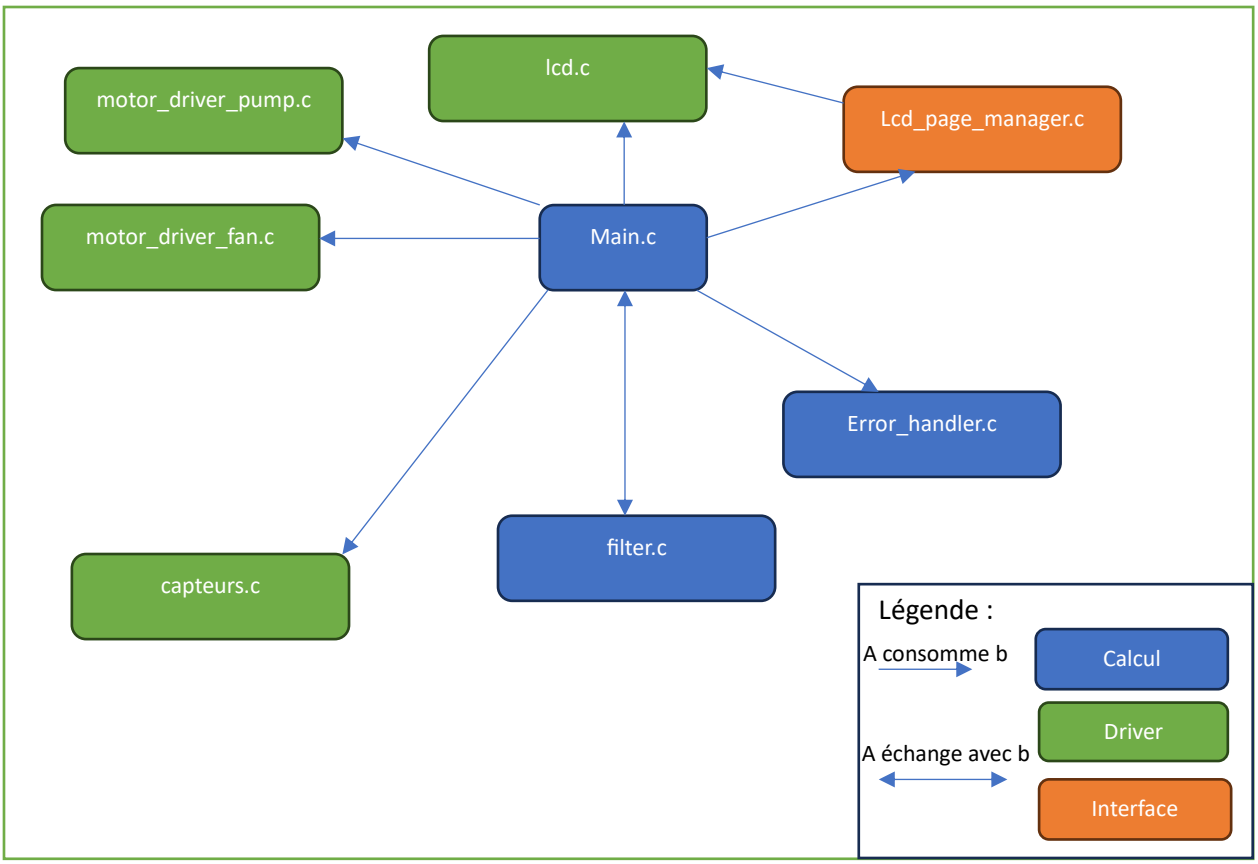


Figure 12 Control Flow

9.2.1 Structure du projet

Afin de faciliter le travail collaboratif et la maintenance, le code a été découpé en plusieurs modules :

- Couche d'Abstraction Matérielle (HAL) : Utilisation des bibliothèques de STMicroelectronics pour configurer les registres des périphériques (ADC, PWM, GPIO).

- Drivers (lcd.c/h, capteurs.c/h, motor_driver_fan.c/h, motor_driver_pump.c/h). Ces fichiers contiennent les fonctions de bas niveau. Par exemple, le driver LCD gère le protocole de communication 4 bits et l'initialisation du contrôleur HD44780, masquant la complexité pour le programmeur principal.

- Gestionnaire d'Interface (lcd_page_manager.c/h) : Ce module gère la machine d'états de l'afficheur (les 5 menus spécifiés) et le rafraîchissement sélectif de l'écran pour éviter les scintillements.

Le détail de chaque fonction implémentée dans chaque unité logicielle est donné ci-dessous.

9.2.1.1 main.c

Gère la boucle principale.

Main -

- Initialisation
- Lecture capteurs
- Prise de décision
- Affichage
- Monitoring

HAL_GPIO_EXTI_Rising_Callback - Fourni la fonction d'interruption des boutons-poussoirs.
ReadADC_Channel - Fourni la fonction de lecture ADC.
LED_BLINK - Fourni la fonction de clignotement d'un LED.

9.2.1.2 motor_driver_pump.c

Fourni les services pour activer/désactiver et retourner l'état de la pompe.

Pump_Init - initialisation de la pompe

Pump_SetSpeed - Définit la vitesse de la pompe en %

Pump_Stop - Arrêt de la pompe

Pump_IsOn - Retourne l'état de la pompe (actif/éteint)

9.2.1.3 motor_driver_fan.c

Fourni les services pour activer/désactiver et retourner l'état du ventilateur.

Motor_On - Activation du ventilateur

Motor_Off - Arrêt du ventilateur

Motor_IsOn - Retourne l'état du ventilateur

9.2.1.4 capteurs.c

Assure la conversion Analogique/ numérique pour les données mesurées.

CAPTEURS_ComputeTemperature - Convertit la valeur brute de l'ADC en température Celsius

CAPTEURS_ComputeHumidity - Convertit la valeur brute de l'ADC en pourcentage d'humidité

9.2.1.5 filter.c

Fourni le service de filtre numérique à moyenne glissante pour supprimer les valeurs aberrantes et éviter le clignotement intempestif de l'écran LCD.

filter_init - Initialisation du Filtre numérique avec valeur de départ

filter_update - Filtre numérique à moyenne glissante

9.2.1.6 lcd.c

C'est le cœur de l'affichage LCD, il fourni plusieurs services qui permettent l'initialisation de l'écran, l'affichage de donnée, le rafraîchissement.

lcd_init - Initialise l'écran LCD (Mode 4 bits, curseur off, auto-incrément).

lcd_send_commande - Envoi les commandes propres à l'écran LCD

lcd_show_carac - Affiche un caractère ASCII sur le LCD.

lcd_show_txt - Affiche une chaîne de caractères à la position actuelle.

LCD_PrintFloat - Affiche un flottant en utilisant un sprintf

lcd_position_cursor - Positionne le curseur à une coordonnée précise. COL (0-15) et row (0-1)

lcd_clear - Efface l'écran complet et repositionne le curseur en (0,0).

lcd_print - Affiche ligne et colonne

lcd_print_int - Affiche un entier sur le LCD.

lcd_print_float_1 - Affiche un flottant sur le LCD sans sprintf

9.2.1.7 lcd_page_manager.c

Permet de faire l'interface entre le cœur LCD et la logique du main.

lcd_page_1 – Logique du menu 1 de l'écran LCD

lcd_page_2 - Logique du menu 2 de l'écran LCD

lcd_page_3 - Logique du menu 3 de l'écran LCD

lcd_page_4 - Logique du menu 4 de l'écran LCD

lcd_page_5 - Logique du menu 5 de l'écran LCD

9.2.1.8 error_handler.c

Assure la surveillance du système, Il détecte les pannes matérielles (capteurs débranchés) ou les pannes d'actionneurs (ventilateur bloqué, réservoir vide) par une analyse temporelle.

error_handler_init - Initialisation de l'error handler

error_handler_update - Analyse les mesures et déclenche les alertes si nécessaire.

error_handler_has_error - Génération des codes erreur

9.2.2 Optimisation des ressources

Une contrainte majeure de notre microcontrôleur STM32G0 est la gestion de la mémoire Flash et RAM.

Affichage optimisé : Pour éviter d'inclure la bibliothèque standard stdio.h (très gourmande avec sprintf), nous avons implémenté notre propre fonction lcd_print_int.
Cette fonction décompose les entiers en caractères ASCII par des opérations de division et modulo, réduisant drastiquement l'empreinte mémoire du projet.

Attentes non-bloquantes : Au lieu d'utiliser HAL_Delay(), nous avons privilégié une fonction lcd_wait_ms basée sur HAL_GetTick().
Cela permet à la CPU de ne pas être totalement figée durant les temps de réponse du LCD, laissant la priorité aux interruptions critiques.

9.2.3 Logique de contrôle-commande

L'implémentation des lois de régulation a été réalisée au sein de la boucle principale (main.c) :

- Traitement ADC : Les données brutes des capteurs sont lissées par un algorithme de moyenne glissante. Cela élimine les pics de bruit électronique lors du démarrage de la pompe.

- Génération PWM : Le pilotage de la pompe utilise le Timer 16 en mode PWM. Le rapport cyclique (Duty Cycle) est calculé dynamiquement :

$$Duty_Cyle = (Seuil - Mesure) / K$$

  Cette approche permet un arrosage progressif, contrairement à un simple pilotage binaire.

9.2.4 Gestion des erreurs

Le module error_handler surveille en permanence la cohérence entre la commande et la mesure.

Si une dérive anormale est détectée (ex: humidité stagnante alors que la pompe est à 100% de puissance), le logiciel lève un FLAG d'erreur, bascule la LED de statut en mode "Défaut" et affiche le diagnostic sur la page 5 du LCD.

9.3.Tests et validation

Pour valider la conformité du système Greenhouse, nous avons mis en place une procédure de test rigoureuse.
Chaque test vise à vérifier un ou plusieurs System Requirements (SR) définis lors de la phase de conception.

A FAIRE !!! PRENDRE DES PHOTOS pour PREUVE de chaque test

Les tests sont présentés dans les sections 9.3.x.
Synthèse des résultats :

Exigence	Libellé	Résultat	Observations
SR-GH-001/002	Acquisition T° / Hum	OK	Précision conforme après filtrage
SR-GH-004/005/006/007	Régulations	OK	Hystérésis et PWM opérationnels
SR-GH-003/008/009	Interactivité	OK	Changement de seuil fluide
SR-GH-010	Mode manuel	OK	Mode manuel validé
SR-GH-011	Diagnostic	OK	Détection de panne validée

9.3.1 Validation de la chaîne d'acquisition (SR-GH-001, 002)

L'objectif est de vérifier la précision des capteurs et le bon fonctionnement du filtrage numérique.

Protocole : Comparaison des valeurs affichées sur le LCD avec les valeurs lues en interne STM32.

Grâce à la fonction UART, la console interne à STM X_IDE permet d'afficher les valeurs brutes, traitées puis filtrées des différents capteurs. Elles sont comparées aux valeurs lues sur l'écran LCD.

Elles sont également comparées à une seconde source matérielle (thermomètre et hygromètre).

Résultat : Grâce au filtre à moyenne glissante, les valeurs sont stables. L'acquisition est validée.

9.3.2 Validation des boucles de régulation (SR-GH-004, 005, 006, 007)

Nous avons testé la réactivité des actionneurs face à des variations de l'environnement.

Test Température : En approchant une source de chaleur du TMP235, le ventilateur s'active dès le franchissement du seuil. L'arrêt se fait correctement après refroidissement sous le seuil d'hystérésis.

Test Humidité : En sortant le capteur capacitif du substrat humide, nous avons observé l'accélération progressive de la pompe via le signal PWM, confirmant la loi de commande proportionnelle.

Résultat : Les boucles de régulation sont validées.

9.3.3 Validation de l'IHM et de l'interactivité (SR-GH-003, 008, 009)

Ce test vérifie que l'utilisateur peut piloter la serre sans reprogrammer le code

Protocole : Navigation à travers les 5 menus et modification des seuils via les boutons.

Résultat : Le lcd_page_manager réagit instantanément aux interruptions des boutons. Les nouvelles consignes sont immédiatement prises en compte par les boucles de régulation.

9.3.4 Validation de l'IHM et de l'interactivité (SR-GH-010)

Ce test vérifie que l'utilisateur peut piloter les actionneurs en mode manuel.

Protocole : Aller sur le menu4 de l'écran LCD.

- Lire l'état des actionneurs
- Activer la pompe puis la désactiver
- Activer le ventilateur puis le désactiver
- Activer les deux
- Quitter le menu 4

Résultat : La lecture des actionneurs correspond bien à leur état. Ils s'activent/désactivent séparément mais aussi ensemble. Le système repasse en automatique lors de la transition vers un autre menu.

9.3.5 Test de sécurité et diagnostic (SR-GH-011)

Il s'agit de vérifier que le système "sait" quand il ne fonctionne plus.

Protocole : Simulation de défaillances (une à la fois puis tout en semble) :

- Déconnexion du capteur d'humidité
- Déconnexion du capteur de température
- Consigne de température « inatteignable » pour le système
- Consigne de d'humidité « inatteignable » pour le système

Résultat : Le système est passé par tous les états d'erreur

- "LE SYSTEME FONCTIONNE BIEN" : Aucune anomalie détectée.
- "FAN SYSTEM KO" : Détecté si problème sur le système de régulation de la température.
- "PUMP SYSTEM KO" : Détecté si problème sur le système de régulation de l'humidité

10.Gestion de projet / management

Exigence explicitement demandée dans le cadrage

Organisation de l'équipe

Répartition des tâches

Planning (diagramme de Gantt ou équivalent)

Difficultés rencontrées

Améliorations possibles en management

11.Analyse critique

Ce qui fonctionne bien

Points faibles et limites

Améliorations techniques possibles

Extensions imaginables (IOT, app mobile, dashboard, serre plus grande…)

12. Bilan du projet

13. Conclusion

Fonctionnalités atteintes

Apports techniques et pédagogiques

Perspectives

14. Glossaire

Annexes

Obligatoires selon la note de cadrage :

Schéma de câblage clair(ajouter le mappage des pin de MX)

Notice d'utilisation du système (démarrage, navigation, maintenance)

Autres annexes possibles :

Tableau de composants

Extraits de code

Courbes de test

Datasheets

Specification système

Template

SR-GH-00x
Révision 00
Objet :
Rationale:

Specification

SR-GH-001
Révision 00
Objet : Greenhouse doit pouvoir acquérir la température ambiante.
Rationale: température en °C

SR-GH-002
Révision 00
Objet : Greenhouse doit pouvoir acquérir l'humidité au sol.
Rationale: humidité en %

SR-GH-003
Révision 00
Objet : Greenhouse doit pouvoir être interactif vis-à-vis de l'opérateur.
Rationale: Pas besoin de reconfiguration (nouveau téléchargement), l'opérateur peut saisir les consignes et allumer/éteindre le ventilateur et/ou la pompe.

SR-GH-004
Révision 00
Objet : Greenhouse doit pouvoir monitorer la température en fonction d'une consigne donnée.
Rationale: Consigne de température donnée par l'utilisateur.

SR-GH-005
Révision 00
Objet : Greenhouse doit pouvoir monitorer l'humidité en fonction d'une consigne donnée.
Rationale: Consigne d'humidité donnée par l'utilisateur.

SR-GH-006
Révision 00
Objet : Greenhouse doit pouvoir réguler la température dans la serre.
Rationale:

SR-GH-007
Révision 00
Objet : Greenhouse doit pouvoir réguler l'humidité dans la serre.
Rationale:

SR-GH-008
Révision 00
*Objet : Greenhouse doit pouvoir afficher les **informations** de températures et d'humidité sur une page d'un écran LCD.*
Rationale:

SR-GH-009
Révision 00
*Objet : Greenhouse doit pouvoir afficher les **consignes** de température et d'humidité sur une page d'un écran LCD.*
Rationale:

SR-GH-010
Révision 00
Objet : Greenhouse doit pouvoir afficher l'état courant du ventilateur et de la pompe.
Rationale: Afficher ON ou OFF

SR-GH-011
Révision 00
Objet : Greenhouse doit pouvoir afficher l'état du système
Rationale: Afficher Système OK ou défaut système de pompe ou défaut système de ventilation

Design écran LCD

Menu1 :

Temperature	x°C
Humidite	x%

Menu2 :

Temperature	x°C
Seuil	x°C

Menu3 :

Humidite	x%
Seuil	x%

Menu4 :

Pompe	ON/OFF
Ventilateur	ON/OFF

Menu5 :

LE SYSTEME
FONCTIONNE BIEN

Ou

FAN SYSTEM	OK/KO
PUMP SYSTEM	OK/KO

Notice d'utilisation

La notice d'utilisation du système est disponible au format pdf.



NOTICE.pdf